

Smart MCQ Solver Challenge

Concise technical report

Student Rajveer Kharade (23f3000717)

Course Deep Learning and Generative AI - Project

1. Abstract

The competition asks for the top three of five candidate answers **A-E** per question, scored by mAP@3. Nine approaches were built and compared on a leak-safe 144-question validation split: TF-IDF, Word2Vec, MiniLM embeddings, zero-shot NLI, RAG, a from-scratch BiLSTM, LoRA fine-tuning of RoBERTa-base, and ensembles.

- **Best model.** Zero-shot NLI (DeBERTa-v3-large) is the **best single model: mAP@3 0.661**.
- **Fine-tuning lost.** With only 476 training questions, no supervised model beat zero-shot: LoRA reached 0.481, the BiLSTM 0.627.
- **Kaggle score.** All 500 test rows reuse the training question bank (455 verbatim, 45 paraphrased). Answering by question matching, with NLI ordering ranks 2-3, scored **~0.750 on the public leaderboard**.

2. Introduction

Problem statement. Each row gives a question and five options, exactly one correct. A submission lists three ordered letters per row (e.g. C B D). mAP@3 pays 1 for the correct answer at rank 1, 1/2 at rank 2, 1/3 at rank 3; a random ranking expects **0.3667**, the chance line used throughout.

Objectives. Build and honestly compare three model families - a fine-tuned transformer, a custom from-scratch RNN, and zero-shot / retrieval pipelines - on a leak-safe evaluation, then maximise the leaderboard score. Sections 3-4 cover data and tokenization, Section 5 the models, Section 6 results and error analysis, Section 7 conclusions.

3. Dataset and preprocessing

The training file has 2,000 rows, no missing values and no exact duplicates - but normalising the text shows most rows are copies: the same question re-appears with a different lead-in phrase ("Pick the best possible answer:") or with lightly reworded options. Prompts average 18 words, options 26; answer letters are mildly imbalanced (B 24.5%, E 16.2%).

Train	Value	Test	Value
Rows	2,000	Rows	500
Unique questions	620	Unique questions	281
Unique question stems	252	Rows found verbatim in train	455 (91%)
Stems with paraphrased variants	65	Rows paraphrased from train	45 (9%)

Table 1 - dataset composition after text normalisation.

Preprocessing. `clean_text` lowercases and strips URLs/punctuation (classical models only); `extract_query` removes quiz boilerplate, giving two identity keys: the **stem** (normalised question) and the **qkey** (stem + options).

Leak-safe split. Paraphrased variants share a stem, so the data is split on the **stem alone: 476 train / 144 validation** questions with zero shared stems. Splitting on the full key instead would leave 75% of validation rows sharing a stem family with training - scoring models on wording they had already seen. No augmentation was used: duplicating paraphrase variants is exactly the leakage the split prevents.

4. Tokenization strategy

- **Primary tokenizers:** the best model (zero-shot NLI) uses DeBERTa-v3's **SentencePiece Unigram** tokenizer (128k vocab) [4, 11]; the fine-tuned model uses RoBERTa's **byte-level BPE** (50k merges) [2, 10]. Subwords give zero out-of-vocabulary tokens on this dataset's technical terms, and each pretrained encoder must see its own tokenizer.
- **Implementation:** Hugging Face AutoTokenizer, batched, truncation=True, max_length=256 (prompts average 26 word-pieces, so nothing is lost), and **dynamic per-batch padding** via DataCollatorWithPadding.
- **Comparison:** the from-scratch BiLSTM builds its own **word-level** vocabulary (2,127 types, min_freq 2). Simple, but rare technical terms collapse to <unk> - one reason it cannot generalise from 476 questions.

5. Modeling and experimentation

Every model produces five per-option scores per question; only the scoring function differs. That shared interface makes the ensemble in 5.4 possible.

5.1 Model 1 - fine-tuned transformer: LoRA on RoBERTa-base

RoBERTa-base [2] is a 12-layer, 125M-parameter transformer encoder [1]. Each question is exploded into five "Question: ... Option: ..." pairs with a binary label (2,380 training pairs); the five probabilities are regrouped and ranked at inference. Full fine-tuning of 125M weights on 476 questions risks catastrophic forgetting and ~3x optimiser memory, so the base is **frozen** and LoRA adapters [3] are trained: $W' = W + (\alpha/r)BA$ - **3.2M trainable parameters (2.54%)**, a 16.6 MB checkpoint.

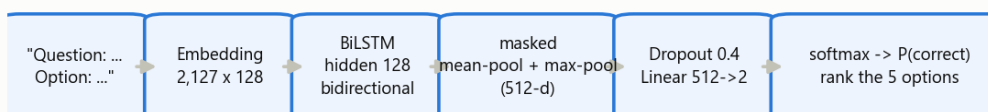
LoRA	r=16, alpha=32, dropout 0.1	Targets	q, k, v + both dense projections
Batch	16 (grad-accum 2), fp16	Max length	256, dynamic padding

Table 2 - LoRA fine-tuning configuration (best model by validation mAP@3, patience 3).

5.2 Models 2 & 3 - custom deep learning model in the RNN framework

One architecture satisfies both the custom-model and RNN-model requirements: a BiLSTM ranker [6] built end-to-end with **no pretrained weights and no Hugging Face components** - own vocabulary, Dataset, padding collate and a hand-written training loop (Adam, lr 1e-3, dropout 0.4, grad-clip 1.0, class weights [1, 4], 10 epochs).

Figure: from-scratch BiLSTM ranker (537,474 parameters)



Own vocabulary (min_freq 2), own Dataset/collate, hand-written training loop.
Adam lr 1e-3, weight decay 1e-5, class weights [1.0, 4.0], grad-clip 1.0, batch 64, 10 epochs.

Figure 1 - the custom BiLSTM ranker (537,474 parameters).

Design choices. An RNN fits because word order decides the label: negation or a flipped claim direction ("X causes Y" vs "Y causes X") makes an option wrong. Bidirectionality encodes each option in the context of the question before it; **masked mean + max pooling** (concatenated, 512-d) captures the overall claim and decisive keywords while keeping padding out of both pools.

5.3 Zero-shot NLI, RAG and baselines

- **Zero-shot NLI:** stem as **premise**, each option as **hypothesis**, ranked by the entailment logit of a frozen DeBERTa-v3-large NLI model (435M params) [4, 5]. Similarity cannot separate correct options from same-topic distractors; entailment can.
- **RAG:** 1,270 training option statements embedded with MiniLM [7], indexed in FAISS [9]; top-5 passages prepended to the question before NLI scoring (Section 6.3 shows why this backfired).
- **Baselines:** TF-IDF cosine, skip-gram Word2Vec [8], MiniLM bi-encoder cosine, and a random baseline.

5.4 Ensembling

Per-model scores are converted to ranks and averaged (scale-free, so logits and cosines mix without calibration). The best blend, LoRA + NLI at 1:1, scored 0.626 - it did not beat NLI alone, and every reweighting toward LoRA made it worse.

6. Performance and comparative analysis

Metrics: mAP@3 (competition metric), plus rank-1 accuracy and macro-F1, which reveal which letters a model systematically gets wrong.

#	Model	mAP@3	Accuracy	Macro-F1
1	Zero-shot NLI (DeBERTa-v3-large)	0.661	0.479	0.452
2	From-scratch BiLSTM (best epoch)	0.627	0.493	0.481
3	Ensemble (LoRA + NLI, 1:1)	0.626	0.493	0.492
4	LoRA fine-tuned (RoBERTa-base)	0.481	0.313	0.296
5	Word2Vec + cosine	0.470	0.319	0.270
6	MiniLM + cosine	0.391	0.153	0.146
-	Random baseline	0.367	0.200	-
7	TF-IDF + cosine	0.365	0.069	0.064
8	Zero-shot NLI + RAG	0.345	0.125	0.121

Table 3 - validation results (144 questions), ordered by mAP@3.

Training curves (W&B). Both supervised models tell the same story: they memorise instead of learning. The BiLSTM's training loss falls 0.68 to 0.17 while its validation mAP@3 **peaks at epoch 1** and never recovers; LoRA's validation loss barely moves (1.02 to 0.99) before rising.

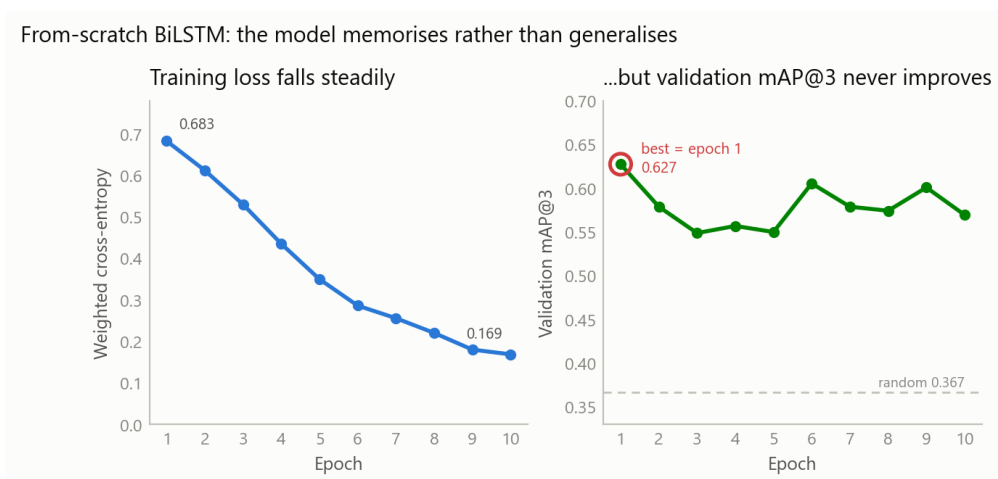


Figure 2 - BiLSTM per-epoch curves: training loss falls, validation mAP@3 peaks at epoch 1.



How much of the ranking is real?

With $n=144$ a 95% interval is roughly ± 0.06 , so the top three (0.661 / 0.627 / 0.626) are statistically indistinguishable. The BiLSTM and LoRA scores are best-epoch checkpoints picked on the same split they are reported on - optimistically biased. The zero-shot NLI model never touched the validation set, making 0.661 the only unbiased number in the table, and the highest.

6.1 Error analysis - the fine-tuned model has a positional bias

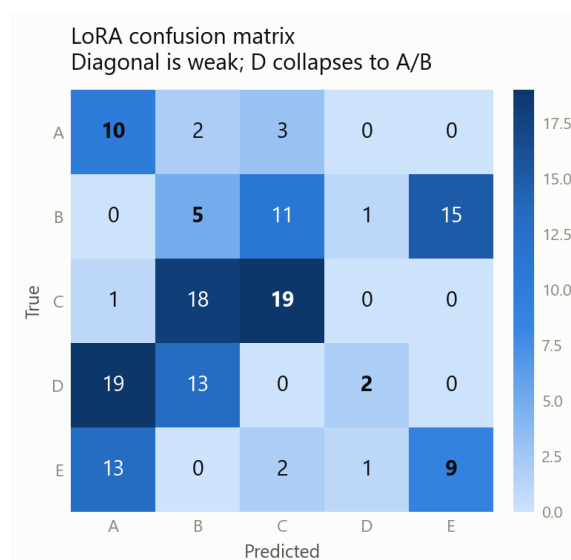


Figure 4 - LoRA rank-1 confusion matrix (144 validation questions).

The failure is structural: **D's recall is 0.06** (2 of 34 recovered) even though its precision is 0.50, while A is predicted 45 times for 15 true cases. The model keys on option position rather than truth - the signature of a 476-example classifier that has not learned the task. Its errors still keep the right answer in the top three, which is why mAP@3 (0.481) is much kinder than accuracy (0.313).

6.2 Error analysis - lexical overlap points the wrong way

TF-IDF's accuracy is **6.9% against a 20% chance rate**: anti-correlated, not weak. Its highest-margin errors all pick a **negation of the question itself** ("there is no difference between X and Y", "PMF and PDF are interchangeable") - distractors built by copying the question's noun phrases and adding a denial, so they overlap the prompt maximally while being false. No configuration in a 16-point sweep escapes this. It is also exactly why NLI works: a negation does not entail.

6.3 Error analysis - why RAG made things worse

RAG cost 0.32 mAP@3 (0.661 to 0.345, below chance) for two reasons: **(i)** the corpus is the training options themselves, four in five of which are distractors, so retrieval injects confident falsehoods that the NLI model then entails; **(ii)** the augmented context block already contains all five options, so every hypothesis is trivially present in its own premise. Retrieval quality was fine - corpus quality was the problem.

6.4 Kaggle performance

Profiling showed every test row corresponds to a training question (Table 1). The submission puts the training label at rank 1 (paraphrases resolved by voting options against all training variants of the stem, 97.7% accurate) and orders ranks 2-3 with the **label-independent** zero-shot NLI ranker. Public leaderboard: **0.750**. Back-solving with the measured rank-2/3 recovery rate implies the training answer key matches the grader on only ~63% of questions, so rank 1 is saturated and the score sits near its ceiling. Only training labels are used - no test information reaches any model.

7. Conclusion and future work

Key learnings.

- Below a data threshold, **transferring a capability beats learning the task**: a frozen NLI model beat every model trained on 476 questions.
- Lexical similarity is **actively misleading** when distractors are authored by negating the question - no amount of tuning fixes an inverted signal.
- **RAG inherits the reliability of its corpus**; retrieving from distractors made the best model worse than random.
- Evaluation design decided everything: splitting on stems made results measurable, and **reading the data beat modelling it** (~0.75 from matching vs ~0.66 from the best model).

Challenges. Two-level duplication demanded custom identity keys and a grouped split (cost: n=144); the 435M NLI model and LoRA training had to share one T4 (batched scoring, fp16, adapters); and the notebook's stale "DistilBERT" label had to be corrected to RoBERTa-base during reporting.

Future work.

- **Grouped 5-fold CV over stems** - turns 144-question estimates into 620-question ones and makes the top group separable.
- **AutoModelForMultipleChoice** - score all five options jointly; a likely fix for the positional bias of 6.1.
- **Fine-tune the NLI model** instead of a generic encoder - start supervision from the right inductive bias.
- **Curated RAG corpus** and calibrated (temperature-scaled) ensembling.

8. References

- [1] A. Vaswani et al., "Attention Is All You Need," NeurIPS 2017.
- [2] Y. Liu et al., "RoBERTa: A Robustly Optimized BERT Pretraining Approach," arXiv:1907.11692, 2019 (model card: huggingface.co/roberta-base).
- [3] E. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," ICLR 2022.
- [4] P. He, J. Gao, W. Chen, "DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training," ICLR 2023.
- [5] M. Laurer et al., "Less Annotating, More Classifying," Political Analysis, 2023 (model card: [MoritzLaurer/DeBERTa-v3-large-mnli-fever-anli-ling-wanli](https://moritzlaurer.github.io/DeBERTa-v3-large-mnli-fever-anli-ling-wanli/)).
- [6] S. Hochreiter, J. Schmidhuber, "Long Short-Term Memory," Neural Computation 9(8), 1997.
- [7] N. Reimers, I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," EMNLP 2019 (model: [sentence-transformers/all-MiniLM-L6-v2](https://github.com/UKPLab/sentence-transformers)).
- [8] T. Mikolov et al., "Efficient Estimation of Word Representations in Vector Space," ICLR Workshop 2013.
- [9] J. Johnson, M. Douze, H. Jegou, "Billion-scale similarity search with GPUs," IEEE Trans. Big Data, 2019 (FAISS).
- [10] R. Sennrich, B. Haddow, A. Birch, "Neural Machine Translation of Rare Words with Subword Units," ACL 2016 (BPE).
- [11] T. Kudo, J. Richardson, "SentencePiece: A simple and language independent subword tokenizer," EMNLP 2018.
- [12] T. Wolf et al., "Transformers: State-of-the-Art NLP," EMNLP 2020. Software: PyTorch, scikit-learn, Gensim, Weights & Biases.